

CG Lab Project2: 网格简化 实验报告

袁晨圖 2023K8009929012

 Repo cauphenuny/ucas-graphics

一、实验环境以及运行效果

1.1 编译环境

环境与实验一相同，没有引入新的依赖库

```
1  add_rules("mode.debug", "mode.release")
2  set_languages("c99", "c++20")
3  add_requires("opengl", {system = true})
4  add_requires("glut", {system = true})
5  add_requires("glfw3", {system = true})
6  add_requires("spdlog", {system = true})
7  add_requires("magic_enum")
8
9  target("project1")
10     set_kind("binary")
11     set_warnings("all")
12     add_files("src/basics/*.cpp")
13     add_packages("opengl")
14     add_packages("glut")
15     add_packages("spdlog")
16     add_packages("glfw3")
17     add_packages("magic_enum")
18     if is_plat("macosx") then
19         add_frameworks("Cocoa", "CoreFoundation", "UIKit")
20     end
21
22 target("meshark")
23     set_kind("static")
24     add_files("src/mesh/meshark/src/*.cc")
25     add_includedirs("src/mesh/meshark/include", {public = true})
26     add_includedirs("src/mesh/external/glm", {public = true})
27     add_headerfiles("src/mesh/meshark/include/**/*.h")
28     add_packages("spdlog")
29
30 target("project2")
31     set_kind("binary")
32     add_files("src/mesh/meshark/apps/simplify.cc")
33     add_deps("meshark")
34     add_packages("spdlog")
35     set_rundir("${projectdir}")
```

1.2 运行方法

```
1 xmake run project2 <input obj path> <output obj path> <ratio|num_edges> [-v|-vv]
```

支持输入简化目标比例或目标边数

-v/-vv 控制 logger 输出等级

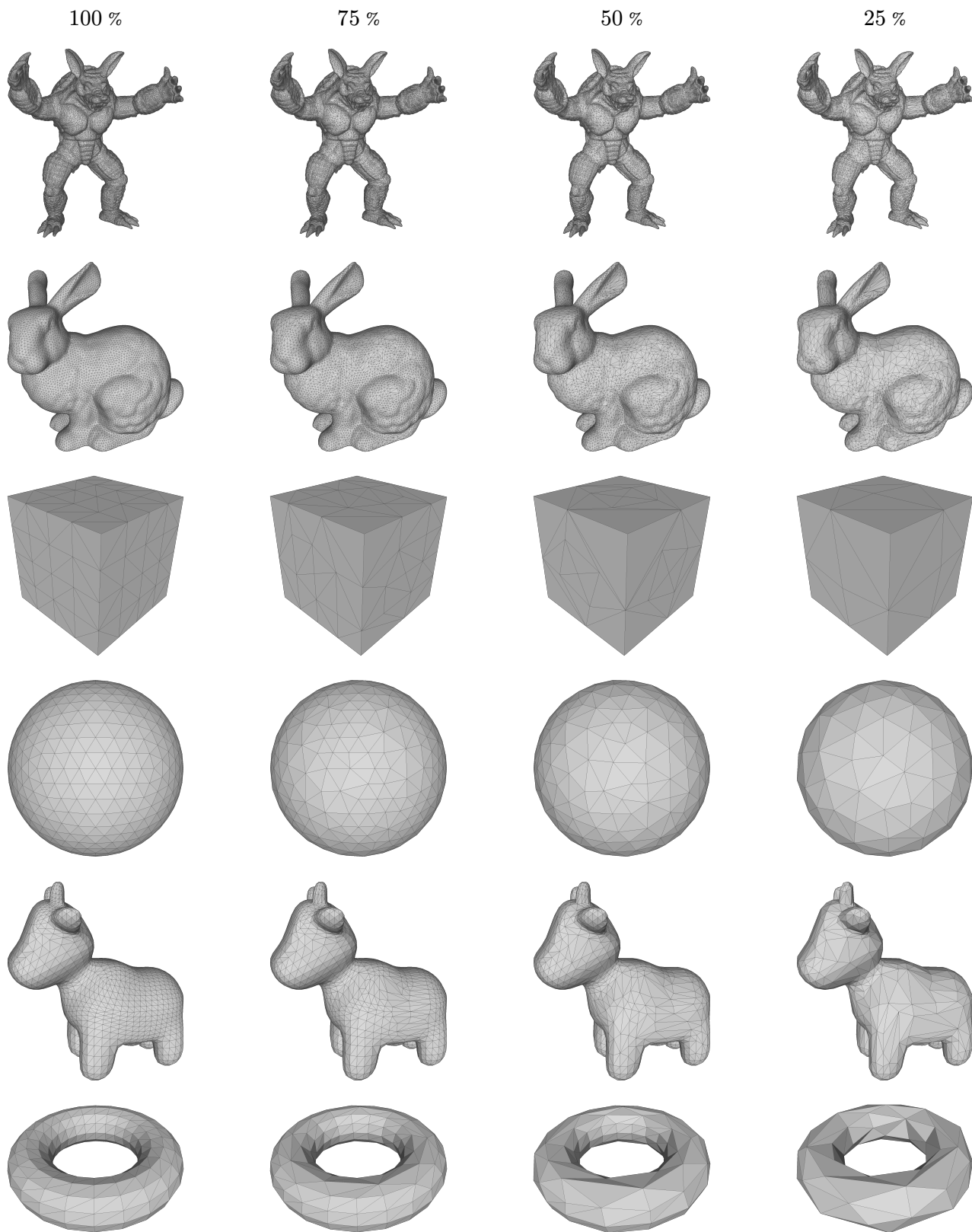
e.g.

```
1 xmake run project2 path/to/obj/sphere.obj path/to/out/sphere25.obj 0.25
```

```
2 xmake run project2 path/to/obj/sphere.obj path/to/out/sphere60e.obj 60 -vv
```

1.3 运行效果

以下展示了不同模型在不同简化率下的效果。其中 100% 表示原始模型（未简化），75%、50%、25% 分别表示保留 75%、50%、25% 的边数。



1.3.1 实验结果分析

从实验结果可以看出：

- 几何形状保持：QEM 算法在简化过程中能够较好地保持模型的整体形状特征。即使在 25% 的简化率下，大部分模型的主要几何特征仍然清晰可见。
- 不同模型的简化效果：
 - 简单几何体（如 sphere、cube_triangle、torus）：简化效果较好，即使在极低简化率下也能保持较好的形状。
 - 复杂模型（如 armadillo、complex_bunny、spot）：在较高简化率（75%、50%）下效果良好，但在 25% 简化率下会出现一些细节丢失，这是预期的结果。
- 算法稳定性：通过面翻转检测等机制，算法能够避免产生拓扑错误和视觉伪影，所有简化结果都保持了正确的网格拓扑结构。

二、代码介绍

2.1 Implementation

2.1.1 Mesh Elements

- VertexElement::OutgoingHalfEdgeIterator::operator++()

首先是实现顶点出边迭代器的 operator++ 方法

这里我重构了一下框架代码，从创建一个新类 OutgoingHalfEdgeRange 并实现

OutgoingHalfEdgeRange::Iterator 改成直接实现一个 iterator，然后通过 std::ranges::subrange 从 start/end iterator 创建 range。

这样的好处一个是写起来方便、避免重复造轮子，二个是这个 std::ranges::subrange 是满足 std::ranges::range concept 的，可以方便地用 ranges 提供的一些基础设施，如果想要自定义类满足这个 concept 的话可能还需要写一些繁琐的东西。

总之实现是这样：

```
1 struct VertexElement::OutgoingHalfEdgeIterator {
2     using difference_type = std::ptrdiff_t;
3     using value_type = HalfEdge;
4
5     OutgoingHalfEdgeIterator& operator++() {
6         it = it->twin->next;
7         if (it == start) it = static_cast<HalfEdge>(nullptr);
8         return *this;
9     }
10    OutgoingHalfEdgeIterator operator++(int) {
11        OutgoingHalfEdgeIterator temp = *this;
12        ++(*this);
13        return temp;
14    }
15
16    HalfEdge operator*() const { return it; }
17
18    bool operator==(const OutgoingHalfEdgeIterator& other) const { return it == other.it; }
19
20    HalfEdge start{nullptr};
21    HalfEdge it{nullptr};
22 };
```

(difference_type 和 value_type 是 std::iterator_traits 推导需要的东西)

```

1 [[nodiscard]] auto VertexElement::outgoingHalfEdges() const {
2     return std::ranges::subrange<OutgoingHalfEdgeIterator>{
3         OutgoingHalfEdgeIterator{he, he}, OutgoingHalfEdgeIterator{he, HalfEdge{nullptr}}};
4 }

```

顺便给 `FaceElement::boundaryHalfEdge` 也重构了一下

2.1.2 Mesh Simplifier

2.1.2.1 Select Edges

1. `MeshSimplifier::computeQuadricMatrix`(Vertex v)

$$\begin{aligned}
 \Delta(v_a \rightarrow v) &= \sum_{p \in \text{plane}(v_a)} (pv^\top)^2 \\
 &= \sum_{p \in \text{plane}(v_a)} (pv^\top)(pv^\top) \\
 &= \sum_{p \in \text{plane}(v_a)} (pv^\top)^\top (pv^\top) \quad (pv^\top \text{ is scalar}) \\
 &= \sum_{p \in \text{plane}(v_a)} vp^\top pv^\top \\
 &= v \left(\sum_{p \in \text{plane}(v_a)} p^\top p \right) v^\top \\
 &= vQ(v_a)v^\top
 \end{aligned}$$

定理 2.1.1

$$Q(v) = \left(\sum_{p \in \text{plane}(v)} p^\top p \right)$$

根据上述公式可得 quadric matrix 的计算方法，即对与该顶点相邻的所有面的平面方程系数向量 p ，计算 $p^\top p$ 的和。

```

1 glm::mat4 MeshSimplifier::computeQuadricMatrix(Vertex v) const {
2     auto result = glm::mat4(0.0f);
3     for (auto h : v->outgoingHalfEdges()) {
4         auto f = h->face;
5         if (!f) continue;
6         auto n = glm::normalize(mesh.normal(f));
7         auto p = mesh.pos(h->tip);
8         Real d = -glm::dot(n, p);
9         glm::vec4 plane(n, d);
10        result += glm::outerProduct(plane, plane);
11    }
12    return result;
13 }

```

2. `MeshSimplifier::computeOptimalCollapsePosition`(Edge e)

合并 $(v_1, v_2) \rightarrow v$ 的误差为：

$$\Delta(v) = \Delta(v_1 \rightarrow v) + \Delta(v_2 \rightarrow v) = v(Q_{v_1} + Q_{v_2})v^\top$$

$$\Delta = (x \ y \ z \ 1) \begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{21} & q_{22} & q_{23} & q_{24} \\ q_{31} & q_{32} & q_{33} & q_{34} \\ q_{41} & q_{42} & q_{43} & q_{44} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \sum_{i=1}^4 \sum_{j=1}^4 v_i v_j q_{ij}$$

求解

$$\frac{\partial \Delta}{\partial x} = \frac{\partial \Delta}{\partial y} = \frac{\partial \Delta}{\partial z} = 0$$

hint: $\frac{\partial(\mathbf{x}^T \mathbf{A} \mathbf{x})}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x}$

可得:

定理 2.1.2

最优点 (x, y, z) 满足

$$\begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{21} & q_{22} & q_{23} & q_{24} \\ q_{31} & q_{32} & q_{33} & q_{34} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = - \begin{pmatrix} q_{14} \\ q_{24} \\ q_{34} \end{pmatrix}$$

```
1 glm::vec3 MeshSimplifier::computeOptimalCollapsePosition(Edge e) const {
2     auto v1 = e->firstVertex();
3     auto v2 = e->secondVertex();
4     glm::mat4 qmat = Q(v1) + Q(v2);
5     glm::mat3 coef(qmat);
6     glm::vec3 rhs(qmat[3][0], qmat[3][1], qmat[3][2]);
7     spdlog::trace(
8         "Computing optimal collapse position for edge {} (det={:.2e}):", e, glm::determinant(coef));
9     if (glm::determinant(coef) < 1e-6) {
10         spdlog::debug("Quadric matrix is singular when collapsing {}, using midpoint", e);
11         return (mesh.pos(v1) + mesh.pos(v2)) * 0.5f;
12     }
13     auto pos = -glm::inverse(coef) * rhs;
14     return pos;
15 }
```

3. MeshSimplifier::computeEdgeCost(Edge e)

根据 QEM 算法, 用 quadric matrix 和 new_pos 计算出 cost

```
1 Real MeshSimplifier::computeEdgeCost(Edge e) const {
2     auto v1 = e->firstVertex();
3     auto v2 = e->secondVertex();
4     auto qmat = Q(v1) + Q(v2);
5     auto merged = computeOptimalCollapsePosition(e);
6     auto homo = glm::vec4(merged, 1.0f);
7     auto cost = glm::dot(homo, qmat * homo);
8     return cost;
9 }
```

2.1.2.2 Collapse Edges

1. MeshSimplifier::removeEdge(), MeshSimplifier::removeVertex()

由于 MeshSimplifier 含有 edge/vertex data, 所以需要在删除边/点时维护一下

```

1 void MeshSimplifier::removeEdge(Edge e) {
2     eraseEdgeMapping(e);
3     edge_collapse_cost.remove(e);
4     mesh.removeEdge(e);
5 }
6
7 void MeshSimplifier::removeVertex(Vertex v) {
8     Q.remove(v);
9     mesh.removeVertex(v);
10 }

```

2. MeshSimplifier::collapseEdge(Edge e)

这是 debug 最久的一块 (

一开始的时候想错了, 与其说是删除 3 条边和它对应的两条半边, 不如说是删掉两个完整的面和它的 boundaryHalfEdge, 后面的这个思路更符合 halfedge-mesh 的结构, debug 也很顺利。

假设 e 的两个端点分别是 v_1, v_2 , 我们将 v_2 合并到 v_1

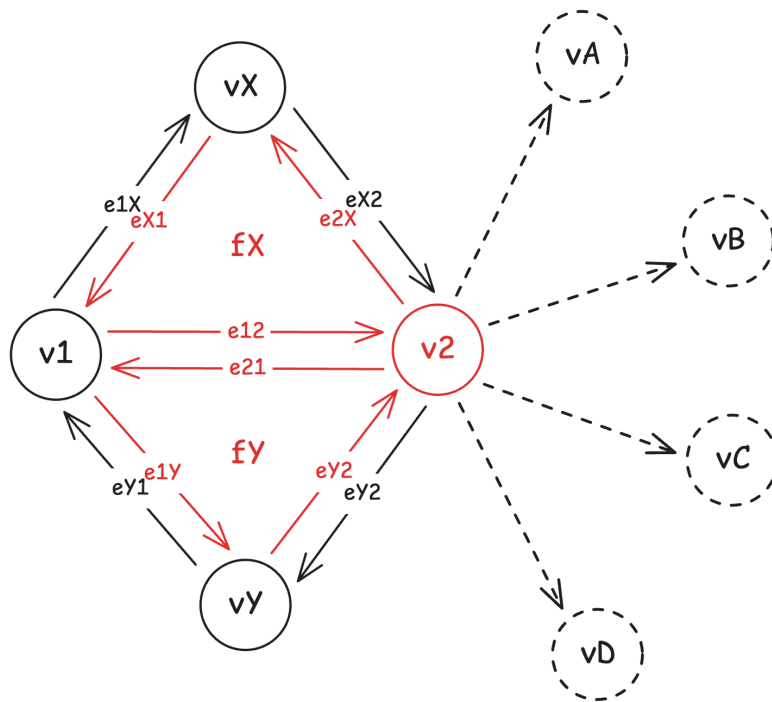


图 1 示意图, 其中红色部分是将被删除的元素

首先取出 v_2 的出入边 (属于 f_x, f_y 的会被删除, 不算)

```

1 auto v2_edges = v2->outgoingHalfEdges()
2     | std::views::filter([=](HalfEdge h) { return !(h->face == fX_ || h->face == fY_); })
3     | std::views::transform([](HalfEdge h) { return std::make_pair(h, h->next->next); });

```

将取出来的出入边重定向到 v_1

```

1 for (auto [out, in] : v2_edges) {
2     out->tail = v1;
3     in->tip = v1;
4 }

```

然后是绑定图中对应的两条黑色实线半边 ($e1X/eX2, eY1/eY2$) 为一条新的边

```

1 auto bind = [this](HalfEdge h1, HalfEdge h2) {
2     h1->twin = h2;
3     h2->twin = h1;
4     removeEdge(h1->edge);
5     h1->edge = h2->edge;
6     h1->edge->halfEdge() = h1;
7 };
8 bind(e1X, e2X);
9 bind(eY1, e2Y);

```

删除 f_x, f_y

```

1 for (auto he : {eX1_, e12_, e2X_}) mesh.removeHalfEdge(he);
2 for (auto he : {eY2_, e21_, e1Y_}) mesh.removeHalfEdge(he);
3 mesh.removeFace(fX_);
4 mesh.removeFace(fY_);

```

由于 v_1, v_x, v_y 的 halfEdge() 可能被删了, 需要重新设定一下

```

1 v1->halfEdge() = e1X;
2 vX->halfEdge() = e2X;
3 vY->halfEdge() = eY1;

```

删除 v_2, e

```

1 removeVertex(v2_);
2 removeEdge(e);

```

3. MeshSimplifier::updateVertexPos(Vertex v, const glm::vec3& pos)

设置点的坐标, 然后更新所有可能被影响到的点的 Q-mat

被更新的 Q-mat 又会造成周围边的 cost 更新

```

1 void MeshSimplifier::updateVertexPos(Vertex v, const glm::vec3& pos) {
2     mesh.setVertexPos(v, pos);
3     Q(v) = computeQuadricMatrix(v);
4     for (auto h : v->outgoingHalfEdges()) {
5         Q(h->tip) = computeQuadricMatrix(h->tip);
6     }
7     for (auto h : v->outgoingHalfEdges()) {
8         for (auto h2 : h->tip->outgoingHalfEdges()) {
9             auto e = h2->edge;
10            updateEdgeCost(e, computeEdgeCost(e));
11        }
12    }
13 }

```

4. MeshSimplifier::collapseMinCoseEdge()

找到最小 cost 的边, 然后尝试 collapse

```

1 MeshSimplifier::MinCostEdgeCollapsingResult MeshSimplifier::collapseMinCostEdge() {
2     auto [cost, min_cost_edge] = *cost_edge_map.begin();
3     spdlog::debug("Collapsing min-cost {} with cost {}", min_cost_edge, cost);
4     if (cost == std::numeric_limits<Real>::infinity()) {
5         spdlog::error("All remaining edges have infinite cost, can not find collapsable edge");
6         exit(1);
7     }
8     auto optimal_pos = computeOptimalCollapsePosition(min_cost_edge);
9     if (mesh.isCollapsible(min_cost_edge, optimal_pos)) {

```

```

10     auto vertex = collapseEdge(min_cost_edge);
11     checkMeshSanity();
12     updateVertexPos(vertex, optimal_pos);
13     return {Edge(), true};
14 } else {
15     return {min_cost_edge, false};
16 }
17 }

```

5. MeshSimplifier::runSimplify(std::variant<int, Real> target)

主循环，计算初始 Q-mat, cost, 然后开始 collapse

```

1 void MeshSimplifier::runSimplify(std::variant<int, Real> target) {
2     for (auto v : mesh.vertices()) Q(v) = computeQuadricMatrix(v);
3     for (auto e : mesh.edges()) {
4         edge_collapse_cost(e) = computeEdgeCost(e);
5         cost_edge_map.insert({edge_collapse_cost(e), e});
6     }
7     checkMeshSanity();
8
9     int target_edges = Match{target}{
10         [](int num) -> int { return num; },
11         [&](Real ratio) -> int { return ratio * num_original_edges; }};
12     spdlog::info("Target: {} edges", target_edges);
13
14     int round = 0;
15     while (mesh.numEdges() > target_edges) {
16         spdlog::info(
17             "Round {} ({} vertices, {} edges, {} faces)", round, mesh.numVertices(),
18             mesh.numEdges(), mesh.numFaces());
19         auto result = collapseMinCostEdge();
20         round++;
21         if (!result.is_collapsible) {
22             auto e = result.failed_edge;
23             updateEdgeCost(e, std::numeric_limits<Real>::infinity());
24             spdlog::warn("Min-cost edge is not collapsable, skip");
25             continue;
26         }
27     }
28 }

```

其中 Match 是一个封装了 std::visit 的小工具，可以方便地对 variant 进行模式匹配

```

1 template <typename... Ts> struct Visitor : Ts... {
2     using Ts::operator()...;
3 };
4
5 template <typename... Ts> Visitor(Ts...) -> Visitor<Ts...>;
6
7 template <typename T> struct Match {
8     T value;
9     Match(T&& value) : value(std::forward<T>(value)) {}
10    template <typename... Ts> auto operator()(Ts&&... params) {
11        return std::visit(Visitor{std::forward<Ts>(params)...}, std::forward<T>(value));
12    }
13    template <typename... Ts> auto operator()(Visitor<Ts...> visitor) {
14        return std::visit(visitor, std::forward<T>(value));
15    }
16 };

```



```
17
18 template <typename T> Match(T&&) -> Match<T&&>;
```

6. GeometryMesh::isCollapsible(Edge e, glm::vec3 target)

这个函数在 HalfEdgeMesh::isCollapsible() 的基础上增加了一个判断，在 bug 分析部分详细描述。

2.2 Refactoring

除了上述的对 iterator 的重构以外，还有一个小重构：

框架中的 VertexData<Type>/EdgeData<Type>/FaceData<Type> 几乎一模一样，用模板类 ElementData<ElementType, T> 抽象出来似乎更好一些：

```
1 template <typename Element, typename Type> struct ElementData {
2     using T = std::conditional_t<std::is_same_v<Type, bool>, std::byte, Type>;
3     // 怎么还有绕过 vector<bool> 的小心思 (笑)
4     ElementData() = default;
5     explicit ElementData(int num_vertices) : data(num_vertices) {}
6     T& operator()(Element elem) { return data[elem->index]; }
7     const T& operator()(Element elem) const { return data[elem->index]; }
8     void append(const T& t) { data.push_back(t); }
9     void remove(Element elem) {
10         if (elem->index == data.size() - 1) {
11             data.pop_back();
12             return;
13         }
14         std::swap(data[elem->index], data.back());
15         data.pop_back();
16     }
17     size_t size() const { return data.size(); }
18
19 private:
20     std::vector<T> data{};
21 };
22
23 template <typename Type> using VertexData = ElementData<Vertex, Type>;
24 template <typename Type> using EdgeData = ElementData<Edge, Type>;
25 template <typename Type> using FaceData = ElementData<Face, Type>;
```

2.3 Utility / Debug Tools

这里是一些相对不那么重要的工具代码

1. MeshSimplifier::checkMeshSanity()

用来检查 halfedge-mesh 的完整性，debug 过程中非常有用，可以迅速发现问题

```
1 void MeshSimplifier::checkMeshSanity() const {
2     if (spdlog::default_logger()->level() >= spdlog::level::info) return;
3     spdlog::debug("Checking Sanity...");
4     assert(mesh.numEdges() == edge_collapse_cost.size());
5     assert(mesh.numVertices() == Q.size());
6     assert(mesh.numEdges() * 2 == mesh.numHalfEdges());
7     assert(mesh.numFaces() * 3 == mesh.numHalfEdges());
8
9     for (auto he : mesh.halfEdges()) {
10         assert(he->twin->twin == he);
11         assert(he->next->tail == he->tip);
12         assert(he->edge->halfEdge() == he || he->edge->halfEdge() == he->twin);
13         assert(he->face->halfEdge());
```

```

14     }
15
16     int sum_of_degrees = 0;
17     for (auto v : mesh.vertices()) {
18         int degree = 0;
19         for (auto h : v->outgoingHalfEdges()) {
20             assert(h->tail == v);
21             degree++;
22         }
23         sum_of_degrees += degree;
24     }
25     if (sum_of_degrees != mesh.numEdges() * 2) {
26         spdlog::error("sum_of_degrees = {}, expected = {}", sum_of_degrees, mesh.numEdges() * 2);
27         // diagnostic: find half-edges not present in any vertex outgoing list
28         std::vector<char> seen(mesh.numHalfEdges(), 0);
29         for (auto v : mesh.vertices()) {
30             for (auto h : v->outgoingHalfEdges()) {
31                 if (h) seen[h->getIndex()] = 1;
32             }
33         }
34         std::vector<int> missing;
35         for (auto he : mesh.halfEdges()) {
36             if (!seen[he->getIndex()]) missing.push_back(he->getIndex());
37         }
38         spdlog::error("missing half-edges (count={}): {}", missing.size(), missing);
39         for (int idx : missing) {
40             auto he = mesh.halfEdge(idx);
41             spdlog::error(
42                 "HalfEdge({}) tail={}, tip={}, twin={}, next={}, edge={}, face={}", idx, he->tail,
43                 he->tip, he->twin ? he->twin->getIndex() : -1, he->next ? he->next->getIndex() : -1,
44                 he->edge, he->face);
45         }
46     }
47     assert(sum_of_degrees == mesh.numEdges() * 2);
48
49     for (auto e : mesh.edges()) {
50         assert(e->halfEdge()->edge == e);
51         assert(e->halfEdge()->twin->edge == e);
52     }
53
54     for (auto f : mesh.faces()) {
55         for (auto h : f->boundaryHalfEdges()) {
56             if (h->face != f) {
57                 spdlog::error("HalfEdge {} has face {}, expected {}", h, h->face, f);
58                 spdlog::error("Face {} boundary half-edges: {:?}", f, f->boundaryHalfEdges());
59             }
60             assert(h->face == f);
61         }
62     }
63 }

```

2. struct fmt::formatter<T>

写了一些给打 log 用的 formatter，这里就不放进来了，代码见 `mystl/fmt.h`

3. MeshElement::repr(), MeshElement::str()

写了这两个函数，把 formatter 暴露给 gdb/lldb，不然 debug 的时候没法复用 formatter 的功能。（名字抄自 python 的 `__repr__`/`__str__`）

```

1  template <typename Type> struct ToString : StaticPolymorphism<Type> {
2      [[gnu::used, gnu::noinline, gnu::visibility("default")]] // for using it in gdb/lldb
3      std::string repr() const {
4          static_assert(fmt::formattable<Type>, "type must be formattable by fmt");
5          // use debug format if available
6          if constexpr (requires { fmt::format("{:?}", this->derived()); }) {
7              return fmt::format("{:?}", this->derived());
8          } else {
9              return fmt::format("{}", this->derived());
10         }
11     }
12     [[gnu::used, gnu::noinline, gnu::visibility("default")]]
13     std::string str() const {
14         static_assert(fmt::formattable<Type>, "type must be formattable by fmt");
15         return fmt::format("{}", this->derived());
16     }
17 };

```

三、 Bug 分析

1. 简化的点位置不对

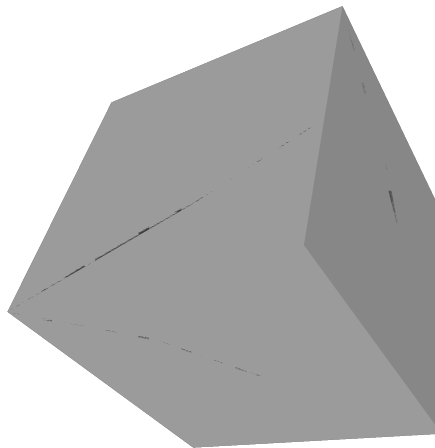
现象：简化后的点位置不对，简化结果不理想，网格形状有很大的变化。

分析：简化时新点的位置计算错误，可能是 Q-matrix 计算错误或者边 cost 没有正确更新。

解决方案：重新分析了一下更新的逻辑，发现之前更新时没有考虑两步的影响，只考虑了当前节点的 Q-mat 和直接相邻的边的 cost，实际上要更新所以相邻点的 Q-mat，更新所有二阶相邻边的 cost。修改后简化结果符合预期。

2. 简化出来的网格有伪影

这是简化 25% 的 cube，可以看到网格中有黑色的条形洞



分析：既然通过了 checkMeshSanity(), 说明拓扑结构没有问题，应该是几何位置导致的，二分查找了造成问题的那一步简化，结果如下：

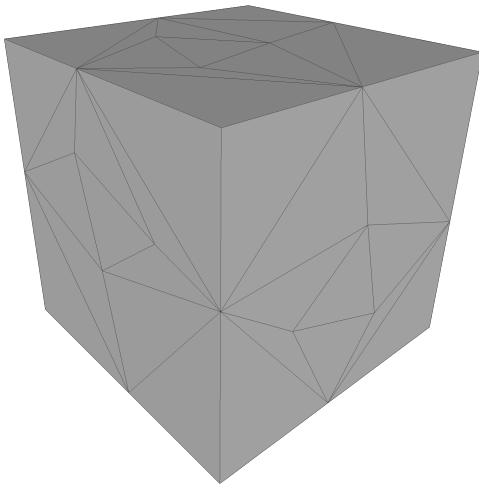


图 2 简化前

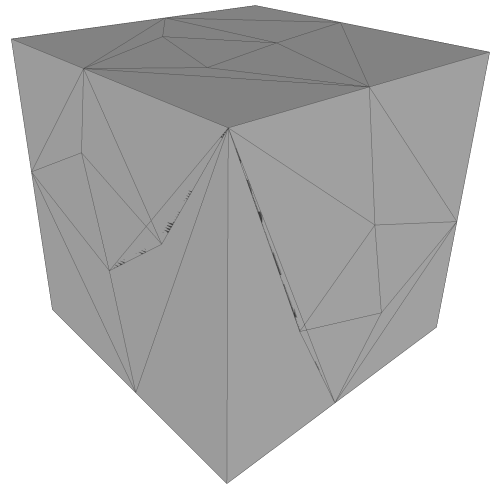


图 3 简化后

可以看到是简化造成了一些面的方向翻转了

解决：新增一个方法 `GeometryMesh::isCollapsible`(Edge e, glm::vec3 target)，在拓扑结构判断的基础上，判断新位置会不会造成面翻转，如果会，那么无法 collapse

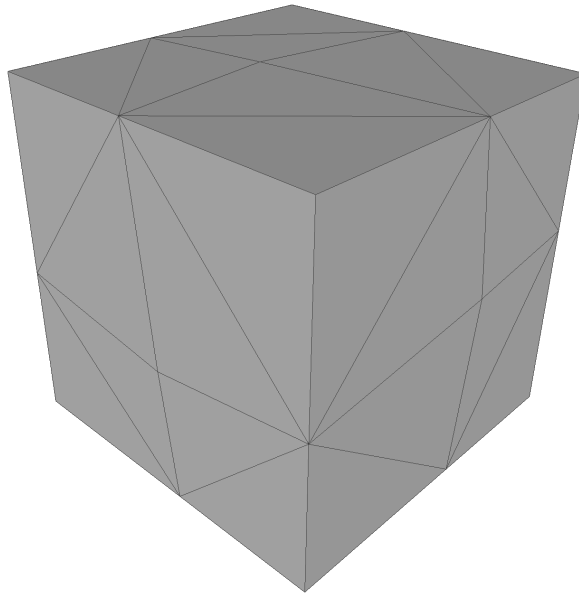
首先获取与一个点相邻的面片（不包含另一个点的，因为这个面会被删），然后在每一个面片中计算新位置与面片另外两个点的叉积，得到新的法线方向，和原法线方向做点积，如果小于 0 就说明翻转了。

```

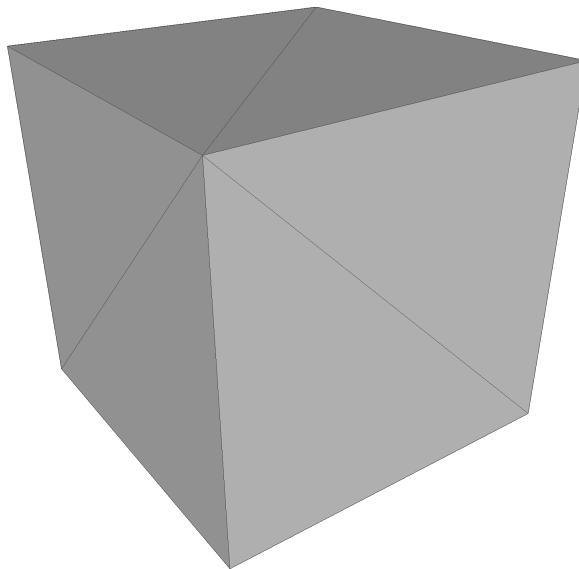
1  bool GeometryMesh::isCollapsible(Edge e, glm::vec3 target) const {
2      if (!HalfEdgeMesh::isCollapsible(e)) return false;
3      auto v1 = e->firstVertex();
4      auto v2 = e->secondVertex();
5      namespace rv = std::ranges::views;
6      auto check = [=, this](Vertex v1, Vertex v2) {
7          // clang-format off
8          auto halfedges = v1->outgoingHalfEdges()
9              | rv::filter([v2](HalfEdge h){ return h->tip != v2;});
10         // clang-format on
11         for (auto he : halfedges) {
12             auto norm = normals(he->face);
13             auto va = he->tip;
14             auto vb = he->next->tip;
15             auto cross = glm::cross(pos(va) - target, pos(vb) - target);
16             if (glm::length(cross) < 1e-6) {
17                 spdlog::warn("Collapse {} would result in degenerate face", e);
18                 return false; // collapse to line
19             }
20             auto new_norm = glm::normalize(cross);
21             if (glm::dot(norm, new_norm) < 1e-3) {
22                 spdlog::warn("Collapse {} would invert face {}", e, he->face);
23                 return false; // face inverted
24             }
25         }
26         return true;
27     };
28     return check(v1, v2) && check(v2, v1);
29 }

```

加上这个判断后没有伪影问题了，以下是 25% 的效果：



可以看到没有伪影了，网格也规则了很多
这是最简的 18 条边的网格，依旧正常



四、总结与体会

这次实验是图形学课程中代码量比较大的一个作业，收获很多。通过实现 QEM 网格简化算法，我对网格数据结构和几何处理有了更深入的理解。

首先给框架一个好评！看起来很爽，写的时候也很舒服

半边表数据结构是这次实验的基础。虽然概念上不难理解，但实际实现起来还是有不少细节需要注意。特别是边坍塌操作，这是整个算法的核心，需要非常小心地维护所有指针关系，确保拓扑结构的一致性。一开始思路不对的时候 debug 了很久，后来想清楚是应该删除两个面而不是删除边再维护面关系之后，实现就顺利多了。这个过程让我深刻体会到，对于复杂的数据结构操作，先理清思路再动手写代码非常重要。

QEM 算法的数学原理在课上已经讲过了，但真正实现的时候还是遇到了一些问题。比如更新 Q 矩阵和边代价的时候，一开始只更新了直接相邻的，后来发现需要更新二阶相邻的才能保证正确性。还有面翻转的问题，这个 bug 花了不少时间才定位到，最后通过检查新位置会不会导致面法线翻转来解决。

在实现过程中，我还应用了一些有意思的 C++ 编程技巧。比如用 C++20 的 `ranges` 创造基于迭代器的 `range`，比传统的自定义 `range` 类+迭代器写法简洁不少，而且可以方便地使用各种 `std::views` 操作，还通过模板元编程抽象出 `ElementData`，避免了代码重复。

整体来说，这次实验不仅让我掌握了网格简化的理论知识，更重要的是培养了系统性思考和问题解决的能力。通过实现一个完整的算法，从数据结构设计到算法实现，再到 bug 调试，每个环节都有不同的挑战。